



ZAP Release 0.2.2

Smart Contract Security Assessment

March 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Incomplete Verification of JUP_V6 Payload	6
4.2	Informational and Undetermined Issues	8
5	Disclaimer	9



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of ZAP smart contracts, starting on March 17th, 2026, and concluding on March 26th, 2026.

Project Overview

Meteora ZAP Release 0.2.2 introduces a pool aware liquidity handler abstraction for DAMM v2 with separate compounding and concentrated paths, rewrites the DAMM v2 zap in balancing and swap estimation flow to use full pool state and handle fee version and compounding fee correctly, adds math helpers and constant updates, fixes single sided DAMM v2 zap in behavior with related test adjustments, and adds a dedicated treasury zap out validation library covering DAMM v2, DLMM and Jupiter Aggregator V6 account and payload parsing, including Jupiter Aggregator V6 whitelisted swap leg checks route plan checks and platform fee input restrictions.

Audit Scope

The assessment scope contains mainly the smart contracts of the zap program for the *Meteora ZAP* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Meteora ZAP
 - Codebase: <https://github.com/MeteoraAg/zap-program>
 - Release 0.2.2: [PR-46](#)
 - Commit Hash: 287ec56ddae718beec0c5fea6cfa40d555b99522

The issues described in this report have been resolved by the following commit:

- Meteora ZAP Release 0.2.2
 - Codebase: <https://github.com/MeteoraAg/zap-program>
 - Commit Hash: 287ec56ddae718beec0c5fea6cfa40d555b99522

We listed the files we have audited below:

- Meteora ZAP Release 0.2.2
 - programs/zap/src/constants.rs
 - programs/zap/src/instructions/ix_zap_in_damm_v2.rs
 - programs/zap/src/lib.rs
 - programs/zap/src/math/mod.rs
 - programs/zap/src/math/utils_math.rs
 - programs/zap/src/state/user_ledger.rs
 - programs/zap/src/utils/damm_v2_utils.rs
 - programs/zap/src/utils/liquidity_handler/compounding_liquidity.rs



- programs/zap/src/utils/liquidity_handler/concentrated_liquidity.rs
- programs/zap/src/utils/liquidity_handler/mod.rs
- programs/zap/src/utils/mod.rs
- protocol-zap/src/constants.rs
- protocol-zap/src/error.rs
- protocol-zap/src/lib.rs
- protocol-zap/src/processors/damm_v2_zap.rs
- protocol-zap/src/processors/dlmm_zap.rs
- protocol-zap/src/processors/jup_v6_zap.rs
- protocol-zap/src/processors/mod.rs
- protocol-zap/src/safe_math.rs
- protocol-zap/src/utils/mod.rs
- protocol-zap/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 1 medium issues
- 0 low issue
- 1 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incomplete Verification of JUP_V6 Payload	Medium	Acknowledged
02	Typo in Comments	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Incomplete Verification of JUP_V6 Payload

Severity: Medium

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Description

In this release, the checks that need to be implemented for integration zap with other protocols were encapsulated into `protocol-zap`, and the payload validation for JUP V6 was hardened:

- For each `input_index` that appears, the cumulative `percent` must `equal 100`
- There must be `only one` terminal output
- `platform_fee_bps` must be 0

However, these hardenings are still incomplete. The following three types of `route_plan` can still pass the current checks:

Case 1: Index 0 is never consumed

Consider this `route_plan` example:

```
[step 1] 1 -> 2 (100%)
```

This type of `route_plan` can satisfy the current two checks, because the only `input_index` that appears is 1, and its cumulative percentage is indeed 100, and there is also only one terminal output, which is 2. However, the actual input of this `route_plan` is at index 0, so it still cannot guarantee that the claimed protocol fee actually enters the Jupiter aggregator. Then the input fee token remains in the token account specified by the operator (route IX) or in the Jupiter aggregator's `program_source_token_account` (shared_accounts_route IX).

Case 2: The outputs still do not fully converge

Consider this `route_plan` example:

```
[step 1] 0 -> 2 (1%)
[step 2] 2 -> 3 (100%)
[step 3] 0 -> 2 (99%)
[step 4] 3 -> 4 (100%)
```

This type of `route_plan` can also pass the current checks, because the cumulative input percentages of index 0, 2, and 3 are all equal to 100, and the only terminal output is 4. However, after index 2 has already been consumed as an input, a new output is later written back to index 2. As a result, after the route ends, the intermediate account at index 2 retains tokens equal to 99% of the input. This means that index 4, as the terminal output



specified in the current validation, does not actually receive output tokens equal in value to 100% of the input.

This type of `route_plan` can also be executed successfully in the Jupiter aggregator, because what the aggregator's `execute_route_plan` returns is the output of the last step, and it does not check whether other `token_step` s still have unconsumed balances.

Case 3: Phantom Input

Consider this `route_plan` example:

```
[step 1] 0 -> 1 (100%)
[step 2] 2 -> 1 (100%)
```

Since there is currently no validation that a non-root input must actually come from a previous step, this type of `route_plan` can also pass the current checks, because the cumulative input percentages of 0 and 2 are both 100, and the only terminal output is 1. But in reality, index 2 is not the output of any previous step.

Impact

The current hardening is still insufficient to guarantee that all inputs are fully consumed during the JUP V6 route and converge to the final output as expected. A malicious operator could still craft a malicious `route_plan` to steal protocol fees.

Recommendation

It is recommended to add at least the following constraints to ensure that the `route_plan` passed into JUP V6 is valid and behaves as expected:

1. Enforce that the cumulative consumption percentage of `input_index = 0` must be exactly 100.
2. Forbid writing output back to an index after that index has already been used as an input.
3. Require that any non-root `input_index` must have been produced by a previous step.

Mitigation Review Log

Meteora Team: We ACK the issue, caused we wont maintain protocol-zap lib in the future, and plan to do a new wrapper program to handle protocol fees. So now it is still relieved on trusted operator.



4.2 Informational and Undetermined Issues

Typo in Comments

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code QA

Case 1:

```
87         // Ensure no platform_fee_bps is 0, so operator can't
           ↳ steal funds by providing their account as
           ↳ platform_fee_account
88     if route_params.platform_fee_bps != 0 {
89         return Err(ProtozolZapError::InvalidZapOutParameters);
90     }
```

[protocol-zap/src/processors/jup_v6_zap.rs#L87-L90](#)

The comment here should be `Ensure platform_fee_bps is 0` .

Case 2:

[protocol-zap/src/utils/token.rs#L18-L18](#): `Adatpted` should be `Adapted` .



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

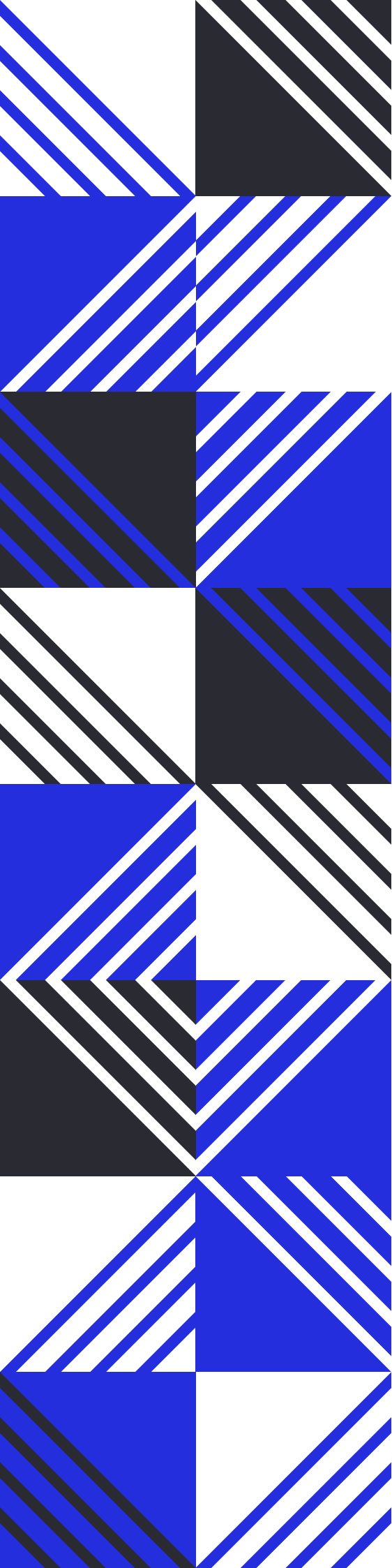
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs